

پروژه ۱: توسعه ماژول سفارشی در CHIPSEC

گزارش نهایی

پارسا علیزاده ۴۰۱۱۱۰۴۹۴

علی الوندی ۴۰۰۱۰۴۷۴۸

آزمایشگاه اینترنت اشیا

دانشگاه صنعتی شریف

بهار ۱۴۰۵

فهرست مطالب

۳	۱	مقدمه
۳	۲	گزارش فنی
۳	۱.۲	مشخصات دستگاه و تنظیمات اولیه
۳	۲.۲	استفاده از نسخه v1.13.20 بجای v2.0.3
۶	۳.۲	اصطلاحات مرتبط با UEFI
۶	۴.۲	بررسی ماژول access_uefispec
۹	۵.۲	آسیب‌پذیری CVE-2022-3432 مرتبط با لپ‌تاپ‌های لنوو
۱۰	۶.۲	دامپ گرفتن از firmware
۱۲	۷.۲	نحوه خوانده شدن متغیرهای EFI
۱۴	۸.۲	متغیرهای تحت تاثیر آسیب‌پذیری لنوو
۱۵	۱.۸.۲	CVE-2021-3971
۱۵	۲.۸.۲	CVE-2021-3972
۱۵	۳.۸.۲	CVE-2022-3430
۱۶	۴.۸.۲	CVE-2022-3431
۱۶	۵.۸.۲	CVE-2022-3432
۱۶	۹.۲	توسعه ماژول غیرفعال برای تشخیص متغیرهای درپشتی
۱۷	۱۰.۲	بررسی CVE-2022-3432 بر روی دستگاه
۱۷	۱۰.۱۰.۲	افزودن متغیر
۱۹	۲۰.۱۰.۲	اجرای ماژول غیرفعال با Secure Boot
۲۲	۱۱.۲	اجرای chipsec در محیط OVMF
۲۶	۱۲.۲	آسیب‌پذیری CVE-2023-48733 مرتبط با OVMF
۲۷	۱۳.۲	نصب سیستم عامل مجازی با QEMU و OVMF
۲۸	۱۴.۲	تلاش اول: جست‌وجوی GUID در حافظه فلش
۲۸	۱۵.۲	تلاش دوم: بررسی قابل بوت بودن شل
۳۰	۱۶.۲	نتایج اجرای ماژول OVMF
۳۳	۳	نتیجه‌گیری
۳۳		منابع

هدف این پروژه مطالعه چارچوب CHIPSEC و توسعه ماژول‌های سفارشی برای آن در جهت بررسی آسیب‌پذیری‌های UEFI بود. در ابتدا نحوه راه‌اندازی این چارچوب، ساختار ماژول‌ها و روش دسترسی آن به اجزای سخت‌افزاری و متغیرهای EFI بررسی شد. سپس با مطالعه چند آسیب‌پذیری واقعی، ماژول‌های سفارشی برای تشخیص آن‌ها توسعه داده شد تا علاوه بر آشنایی با مبانی امنیت firmware، فرآیند تبدیل اطلاعات یک آسیب‌پذیری به یک آزمون عملی در CHIPSEC نیز انجام شود.

دو هدف اصلی پروژه با موفقیت انجام شد. هدف اول بررسی آسیب‌پذیری‌های مرتبط با متغیرهای درپشتی در دستگاه‌های لنوو بود که شامل توسعه یک ماژول غیرفعال برای شناسایی متغیرهای آسیب‌پذیر و اجرای دستی سناریوی بهره‌برداری روی دستگاه فیزیکی می‌شد. هدف دوم آماده‌سازی محیط شبیه‌سازی شده QEMU و OVMF برای اجرای CHIPSEC و بررسی یک پیکربندی ناامن پروژه EDK II مرتبط با Secure Boot و EFI Shell بود. در این بخش نیز با اجرای آزمایش روی نسخه‌های آسیب‌پذیر و تصحیح‌شده، یک ماژول سفارشی برای تشخیص این آسیب‌پذیری پیاده‌سازی و ارزیابی شد.

۲ گزارش فنی

۱.۲ مشخصات دستگاه و تنظیمات اولیه

در مرحله اول ابتدا ماژول‌های پیش‌فرض CHIPSEC بر روی دستگاهی با مشخصات زیر اجرا شده است تا آشنایی لازم با راه‌اندازی CHIPSEC و نحوه کار با آن به دست بیاید.

Device Model	Lenovo IdeaPad Y700-15ISK
Operating System	Fedora Linux 44
CPU	Intel Core i7-6700HQ (Gen 6 Skylake)
Chipset	Intel HM170

جدول ۱: مشخصات دستگاه

برای اجرای موفق CHIPSEC لازم است ابتدا دو تغییر انجام شود. اول اینکه Secure Boot که به طور پیش‌فرض روشن است باید دستی غیرفعال شود تا CHIPSEC بتواند ماژول‌های دلخواه کرنل در حین اجرا اضافه کند. دومین تغییر غیرفعال کردن SELinux است که به طور پیش‌فرض در توزیع Fedora فعال شده است. هر دو تغییر، محدودیت‌های امنیتی دستگاه را کاهش می‌دهند تا CHIPSEC بتواند به درستی ماژول‌ها را اجرا کند.

۲.۲ استفاده از نسخه v1.13.20 بجای v2.0.3

در اولین اجرا از CHIPSEC به خطای زیر بر می‌خوریم.

```
(chipsec) ~/chipsec$ sudo python chipsec_main.py
[sudo] password for parin:
#####
##                                ##
## CHIPSEC: Platform Hardware Security Assessment Framework ##
##                                ##
#####
[CHIPSEC] Version : 2.0/3
[CHIPSEC] Arguments:

INFO: Unknown Platform: VID = 0x1022, DID = 0x1630, RID = 0x00, CVID = 0x000F01
IDF | VID:DID | Vendor | Device
-----
00:00.0 | 1022:1630 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne Root Complex
00:00.2 | 1022:1631 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne IOMMU
00:01.0 | 1022:1632 | Advanced Micro Devices, Inc. | AMD | Renoir PCIe Dummy Host Bridge
00:02.0 | 1022:1632 | Advanced Micro Devices, Inc. | AMD | Renoir PCIe Dummy Host Bridge
00:02.1 | 1022:1634 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne PCIe GPP Bridge
00:02.2 | 1022:1634 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne PCIe GPP Bridge
00:02.3 | 1022:1634 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne PCIe GPP Bridge
00:02.4 | 1022:1634 | Advanced Micro Devices, Inc. | AMD | Renoir/Cezanne PCIe GPP Bridge
00:08.0 | 1022:1632 | Advanced Micro Devices, Inc. | AMD | Renoir PCIe Dummy Host Bridge
00:00.1 | 1022:1635 | Advanced Micro Devices, Inc. | AMD | Renoir Internal PCIe GPP Bridge to Bus
00:14.0 | 1022:7900 | Advanced Micro Devices, Inc. | AMD | FCH SMBus Controller
00:14.3 | 1022:7900 | Advanced Micro Devices, Inc. | AMD | FCH LPC Bridge
00:18.0 | 1022:1448 | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 0
00:18.1 | 1022:1449 | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 1
00:18.2 | 1022:144A | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 2
00:18.3 | 1022:144B | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 3
00:18.4 | 1022:144C | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 4
00:18.5 | 1022:144D | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 5
00:18.6 | 1022:144E | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 6
00:18.7 | 1022:144F | Advanced Micro Devices, Inc. | AMD | Renoir Device 24: Function 7
01:00.0 | 1507:5008 | Sandisk Corp | 
02:00.0 | 10EC:8168 | Realtek Semiconductor Co., Ltd. | RTL8111/8168/8411 PCI Express Gigabit Ethernet Controller
03:00.0 | 10EC:8022 | Realtek Semiconductor Co., Ltd. | RTL8852E0 802.11ax PCIe Wireless Network Adapter
04:00.0 | 1179:011A | Toshiba Corporation | 386 Mfme SSD Controller
```

شکل ۱: خروجی CHIPSEC نسخه 2.0.3

در ابتدای خروجی خطای Unknown Platform آمده است که در ادامه خروجی ذکر شده که بعضی از تست‌ها به همین دلیل اجرا نشده‌اند. به همین خاطر این نسخه از CHIPSEC تنها مازول‌های عمومی را تست کرده و گزارش داده است.

پردازنده دستگاه فعلی Intel(R) Core(TM) i7-6700HQ است که از نسل ششم پردازنده‌های اینتل یعنی Skylake است. همچنین PCH در این دستگاه Intel Corporation HM170 Chipset LPC/eSPI Controller است. به دلیل قدیمی بودن پردازنده و PCH هر دو در آخرین نسخه CHIPSEC قابل تشخیص نبوده‌اند. قدیمی‌ترین نسل پشتیبانی شده در این نسخه Kaby Lake یا نسل هفتم اینتل است.

در CHIPSEC اطلاعات مربوط به پلتفرم‌ها در مسیر /chipsec/cfg/8086/ chipsec/نگه‌داری می‌شود که برای هر پلتفرم به صورت فایل XML است. با بررسی نسخه‌های قدیمی‌تر، متوجه می‌شویم که آخرین نسخه قبل از 2 یعنی نسخه 1.13.2 تنظیماتی برای نسل Skylake و همچنین PCH1XX دارد. با تغییر به این نسخه خطای مربوط به پلتفرم رفع می‌شود.

```

(venv) ➔ chipsec sudo python chipsec_main.py
[sudo] password for parsa:

#####
##                                     ##
## CHIPSEC: Platform Hardware Security Assessment Framework ##
##                                     ##
#####
[CHIPSEC] Version : 1.13.20
[CHIPSEC] Arguments:

[CHIPSEC] OS      : Linux 6.19.12-200.fc43.x86_64 #1 SMP PREEMPT_DYNAMIC Sun Apr 12 15:26:33 UTC 2026 x86_64
[CHIPSEC] Python  : 3.14.3 (64-bit) - Enabled GIL
[CHIPSEC] Helper   : LinuxHelper (/home/parsa/University/aziot/chipsec/chipsec/helper/linux/chipsec.ko)
[CHIPSEC] Platform: Mobile 6th Generation Core Processor Quad Core (Skylake H)
[CHIPSEC] CPUID    : 506E3
[CHIPSEC] VID      : 8086
[CHIPSEC] DID      : 1910
[CHIPSEC] RID      : 07
[CHIPSEC] PCH      : Intel HM170 (100 series) PCH
[CHIPSEC] VID      : 8086
[CHIPSEC] DID      : A14E
[CHIPSEC] RID      : 31

```

شکل ۲: خروجی CHIPSEC نسخه 1.13.2

```

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed          0.229
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed       15:
[+] PASSED: chipsec.modules.common.bios_kbrd_buffer
[+] PASSED: chipsec.modules.common.bios_smi
[+] PASSED: chipsec.modules.common.bios_ts
[+] PASSED: chipsec.modules.common.ia32cfg
[+] PASSED: chipsec.modules.common.memconfig
[+] PASSED: chipsec.modules.common.memlock
[+] PASSED: chipsec.modules.common.remap
[+] PASSED: chipsec.modules.common.rtclock
[+] PASSED: chipsec.modules.common.smm
[+] PASSED: chipsec.modules.common.smm_code_chk
[+] PASSED: chipsec.modules.common.smm_dma
[+] PASSED: chipsec.modules.common.smmr
[+] PASSED: chipsec.modules.common.spd_wd
[+] PASSED: chipsec.modules.common.spi_fdopss
[+] PASSED: chipsec.modules.common.uefi.access_uefispec
[CHIPSEC] Modules information 1:
[#] INFORMATION: chipsec.modules.common.cpu.cpu_info
[CHIPSEC] Modules failed      5:
[-] FAILED: chipsec.modules.common.bios_wp
[-] FAILED: chipsec.modules.common.me_mfg_mode
[-] FAILED: chipsec.modules.common.spi_access
[-] FAILED: chipsec.modules.common.spi_desc
[-] FAILED: chipsec.modules.common.spi_lock
[CHIPSEC] Modules with warnings 4:
WARNING: chipsec.modules.common.cpu.spectre_v2
WARNING: chipsec.modules.common.debugenabled
WARNING: chipsec.modules.common.secureboot.variables
WARNING: chipsec.modules.common.sgx_check
[CHIPSEC] Modules not applicable 6:
NOT APPLICABLE: chipsec.modules.common.cet
NOT APPLICABLE: chipsec.modules.common.cpu.ia_untrusted

```

شکل ۳: خلاصه تست‌های انجام شده

برخلاف اجرای اولیه تست‌های بیشتری بررسی شده‌اند و فقط ۶ ماژول نیازی به بررسی در این پلتفرم نداشته‌اند. در این تست ۱۵ ماژول پذیرفته شده‌اند و ۵ ماژول رد شده‌اند، که این ۵ ماژول می‌توانند نقطه شروعی برای بررسی خطاها و آسیب‌پذیری‌های UEFI باشند.

۳.۲ اصطلاحات مرتبط با UEFI

چه در اسم ماژول‌ها، کد و کامنت آن‌ها، و منابع دیگر اسامی زیر استفاده شده است که نام کامل آن‌ها در ادامه آورده‌ایم.

NVRAM	Non-Volatile Random-Access Memory
MSR	Model Specific Register
SPI Flash	Serial Peripheral Interface Flash
SMM	System Management Mode

۴.۲ بررسی ماژول `access_uefispec`

در این قسمت یکی از ماژول‌های CHIPSEC از نظر پیاده‌سازی و نوع آسیب‌پذیری تست شده بررسی شده است. کد این ماژول در `modules/common/uefi/access_uefispec.py` آمده است. وظیفه این ماژول این است که متغیرهای EFI که در NVRAM نوشته می‌شوند را بررسی کند. به این شکل که طبق UEFI Specification متغیرهای اصلی باید دسترسی‌های به خصوصی داشته باشند. ویژگی‌های مشخص شده برای هر متغیر یک یا چند ویژگی از مجموعه زیر است.

NV	Non-Volatile
BS	Boot Services
RT	Runtime Services
AT	Time-Based Authenticated

این ماژول متغیرهایی که در UEFI Specification مشخص شده‌اند را بررسی می‌کند و اگر ویژگی اشتباهی داشته باشند (مثلا تنظیمات اشتباه در تولید دستگاه انجام شده باشد) خطا می‌دهد.

```

57 class access_uefispec(BaseModule):
58
59 > def __init__(self):...
126
127 > def is_supported(self) -> bool:...
132
133 > def diff_var(self, data1: int, data2: int) -> bool:...
146
147 > def can_modify(self, name: str, guid: str, data: bytes, attrs: int) -> bool:...
168
169 > def check_vars(self, do_modify: bool, attribute: Optional[int] = None) -> int:...
252
253 > def run(self, module_argv: List[str]) -> int:...
265

```

شکل ۴: کلاس `access_uefispec`

هر ماژول به صورت یک کلاس نوشته می‌شود که دو تابع دارد. تابع اول `is_supported` است که در آن ماژول با اطلاعاتی که از CPU، PCH و UEFI دارد بررسی می‌کند که آیا می‌تواند اجرا شود یا خیر. در صورتی که ماژول پشتیبانی بشود، تابع دوم یعنی `run` صدا زده می‌شود. خروجی تابع `run` وضعیت نهایی تست انجام شده توسط ماژول است که برای مثال می‌تواند `passed` یا `failed` باشد.

```
def __init__(self):
    BaseModule.__init__(self)
    self.uefi = UEFI(self.cs)

    nv = EFI_VARIABLE_NON_VOLATILE
    bs = EFI_VARIABLE_BOOTSERVICE_ACCESS
    rt = EFI_VARIABLE_RUNTIME_ACCESS
    ta = EFI_VARIABLE_TIME_BASED_AUTHENTICATED_WRITE_ACCESS

    self.uefispec_vars = {
        # From UEFI Spec Table 11 "Global Variables"
        # Updated to version 2.10 Section 3.3 "Globally Defined Variables"
        # https://uefi.org/sites/default/files/resources/UEFI_Spec_2_10_Aug29.pdf
        # https://uefi.org/specs/UEFI/2.10/03_Boot_Manager.html#globally-defined-variables
        "LangCodes": bs | rt,
        "Lang": nv | bs | rt,
        "Timeout": nv | bs | rt,
        "PlatformLangCodes": bs | rt,
        "PlatformLang": nv | bs | rt,
        "ConIn": nv | bs | rt,
        "ConOut": nv | bs | rt,
        "ErrOut": nv | bs | rt,
        "ConInDev": bs | rt,
        "ConOutDev": bs | rt,
        "ErrOutDev": bs | rt,
        "Boot0001": nv | bs | rt,
        "Boot0002": nv | bs | rt,
        "BootOrder": nv | bs | rt,
```

شکل ۵: ویژگی‌های تعریف شده در ماژول

```

def check_vars(self, do_modify: bool, attribute: Optional[int] = None) -> int:
    res = ModuleResult.PASSED
    vars = self._uefi.list_EFI_variables()
    if vars is None: ...

    uefispec_concern = []
    ro_concern = []
    rw_variables = []

    self.logger.log(['*] Testing UEFI variables ..'])
    for name in vars.keys():
        if name is None:
            pass
        if vars[name] is None:
            pass

        if len(vars[name]) > 1:
            self.logger.log_important(f'Found two instances of the variable {name}.')
        for (off, buf, hdr, data, guid, attrs) in vars[name]:
            self.logger.log(f'[*] Variable {name} ({get_attr_string(attrs)}) Guid {guid} Size {hex(len(data))} ')
            perms = self.uefispec_vars.get(name)
            if perms is not None:
                if perms != attrs:
                    attr_diffs = (perms ^ attrs)
                    extra_attr = attr_diffs & attrs
                    missing_attr = attr_diffs & ~extra_attr
                    uefispec_concern.append(name)
                    if extra_attr != 0:
                        self.logger.log_important(' Extra attributes:' + get_attr_string(extra_attr))
                        if (extra_attr & ~(EFI_VARIABLE_AUTHENTICATED_WRITE_ACCESS |
                            EFI_VARIABLE_TIME_BASED_AUTHENTICATED_WRITE_ACCESS |
                            EFI_VARIABLE_APPEND_WRITE) != 0):
                            res = ModuleResult.FAILED
                    if missing_attr != 0:
                        self.logger.log_important(' Missing attributes:' + get_attr_string(missing_attr))
                    if res != ModuleResult.FAILED:
                        res = ModuleResult.WARNING
            self.result.setStatusBit(self.result.status.VERIFY)

```

شکل ۶: تابع `check_vars`. لیست متغیرهای دستگاه با تابع `self._uefi.list_EFI_variables` گرفته شده است.

```

[x)][ =====
[x)][ Module: Access Control of EFI Variables
[x)][ =====
[*] Testing UEFI variables ..
[*] Variable dbxDefault (BS+RT) Guid 8be4df61-93ca-11d2-aa0d-00e098032b8c Size 0x28c
[*] Variable dbDefault (BS+RT) Guid 8be4df61-93ca-11d2-aa0d-00e098032b8c Size 0x1977
[*] Variable KEKDefault (BS+RT) Guid 8be4df61-93ca-11d2-aa0d-00e098032b8c Size 0x618
[*] Variable PKDefault (BS+RT) Guid 8be4df61-93ca-11d2-aa0d-00e098032b8c Size 0x354
[*] Variable IrsiInfo (BS+RT) Guid 5bce4c83-6a97-444b-63b4-672c014742ff Size 0x10
[*] Variable LoaderDevicePartUUID (BS+RT) Guid 4a67b082-0a4c-41cf-b6c7-440b29bb8c4f Size 0x4a
[*] Variable LoaderInfo (BS+RT) Guid 4a67b082-0a4c-41cf-b6c7-440b29bb8c4f Size 0x14
[*] Variable MokListTrustedRT (BS+RT) Guid 605dab50-e046-4300-abb6-3dd810dd8b23 Size 0x1
[*] Variable SbatLevelRT (BS+RT) Guid 605dab50-e046-4300-abb6-3dd810dd8b23 Size 0x12
[*] Variable MokListXRT (BS+RT) Guid 605dab50-e046-4300-abb6-3dd810dd8b23 Size 0x4c

```

شکل ۷: خروجی اجرای تکی ماژول

۵.۲ آسیب‌پذیری CVE-2022-3432 مرتبط با لپ‌تاپ‌های لنوو

در ماژول نمونه متغیرهای EFI بررسی شده بودند. تغییر دادن بعضی از متغیرها اجازه می‌دهد آسیب‌پذیری‌هایی در فرآیند boot به وجود بیاید یا قابلیت‌هایی مثل Secure boot غیرفعال شود. با جستجو درباره آسیب‌پذیری‌های UEFI به خصوص آسیب‌پذیری‌هایی که دستگاه فعلی را تحت تاثیر قرار می‌دهند (Lenovo ideapad Y700-15ISK) به مورد CVE-2022-3432 [۱] برمی‌خوریم که به طور خاص نسخه 14ISK از همین لپ‌تاپ را تحت تاثیر قرار می‌دهد.

CVE-2022-3432 PUBLISHED [View JSON](#) | [User Guide](#)

Collapse all

Required CVE Record Information

CNA: Lenovo Group Ltd.

Updated: 2023-01-26
Published: 2023-01-23

Description

A potential vulnerability in a driver used during manufacturing process on the Ideapad Y700-14ISK that was mistakenly not deactivated may allow an attacker with elevated privileges to modify secure boot setting by modifying an NVRAM variable.

شکل ۸: آسیب‌پذیری CVE-2022-3432

در این [صفحه \[۲\]](#) توضیحاتی در رابطه با این آسیب‌پذیری و دو آسیب‌پذیری مشابه دیگر آمده است. در هر سه آسیب‌پذیری متغیرهای به خصوصی در زمان تست و تولید دستگاه در NVRAM باقی‌مانده‌اند که با استفاده از آن‌ها می‌توان Secure Boot را غیرفعال کرد.

Lenovo Notebook BIOS Vulnerabilities

Lenovo Security Advisory: LEN-94952

Potential Impact: Disable Secure Boot

Severity: High

Scope of Impact: Lenovo-specific

CVE Identifier: CVE-2022-3430, CVE-2022-3431, CVE-2022-3432

Summary Description:

The following vulnerabilities were reported in Lenovo Notebook BIOS.

CVE-2022-3430: A potential vulnerability in the WMI Setup driver on some consumer Lenovo Notebook devices may allow an attacker with elevated privileges to modify secure boot setting by modifying an NVRAM variable.

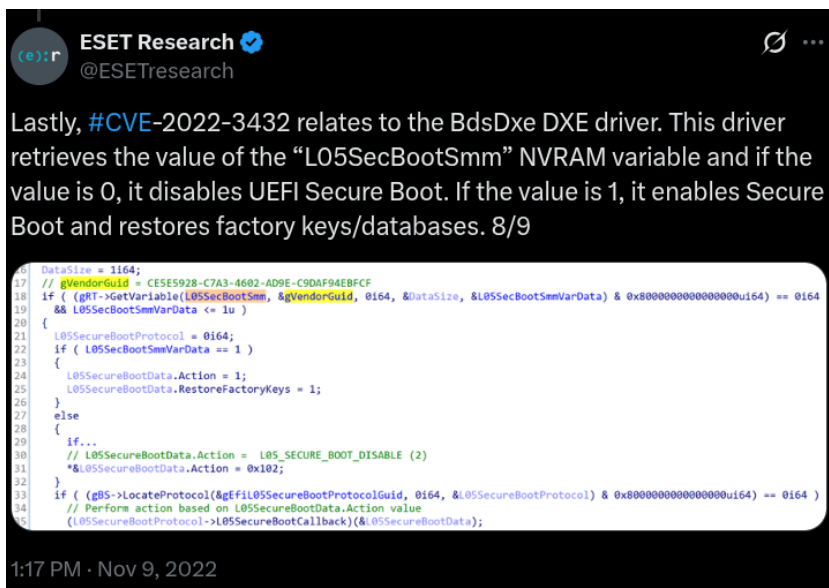
CVE-2022-3431: A potential vulnerability in a driver used during manufacturing process on some consumer Lenovo Notebook devices that was mistakenly not deactivated may allow an attacker with elevated privileges to modify secure boot setting by modifying an NVRAM variable.

CVE-2022-3432: A potential vulnerability in a driver used during manufacturing process on the Ideapad Y700-14ISK that was mistakenly not deactivated may allow an attacker with elevated privileges to modify secure boot setting by modifying an NVRAM variable.

شکل ۹: صفحه Lenovo شامل توضیحات و دستگاه‌های تحت تاثیر

منابع فنی که از این آسیب‌پذیری‌ها وجود دارد یکی [این پست وبلاگ \[۳\]](#) است که مربوط به آسیب‌پذیری قدیمی‌تری ولی مرتبط CVE-2021-3972 است. در این آسیب‌پذیری متغیر ChgBootDxeHook استفاده می‌شود

تا UEFI غیرفعال شود. منبع غیررسمی دیگر از همین آسیب‌پذیری [توییتی از ESET Research](#) [۴] است که متغیرهای تحت تاثیر در این آسیب‌پذیری را توضیح داده است.



شکل ۱۰: توییت ESET Research در توضیح آسیب‌پذیری CVE-2021-3972. با تغییر دادن مقدار L05SecBootSmm به صفر، Secure Boot غیرفعال می‌شود

۶.۲ دامپ گرفتن از firmware

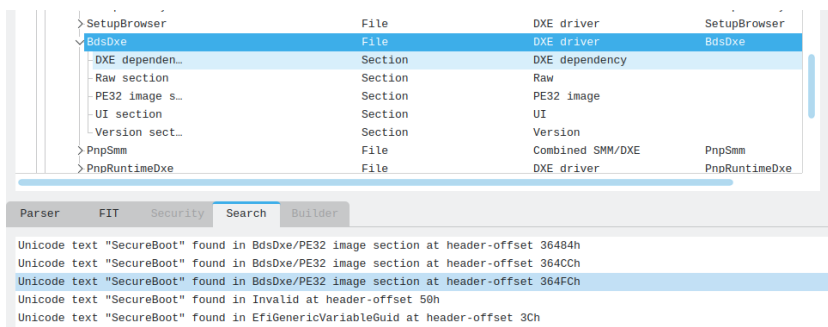
در CHIPSEC دستوری برای دامپ وجود دارد که خروجی آن یک فایل باینری است. این فایل باینری تمام اجزای firmware را شامل می‌شود. در این قسمت می‌بینیم که چگونه می‌توان به این فایل دست پیدا کرد و با استفاده از ابزارهای UEFITools و ghidra به متن firmware دسترسی داشت.

```
[CHIPSEC] Executing command 'spi' with args ['dump', 'bios_backup.bin']

[CHIPSEC] Dumping entire SPI flash memory to 'bios_backup.bin'
[CHIPSEC] it may take a few minutes (use DEBUG or VERBOSE logger options to see progress)
[CHIPSEC] BIOS region: base = 0x00200000, limit = 0x007FFFFF
[CHIPSEC] Dumping 0x00800000 bytes (to the end of BIOS region)
[spi] Reading 0x800000 bytes from SPI at FLA = 0x0 (in 131072 0x40-byte chunks + 0x0-byte remainder)
-----
SPI read complete
[CHIPSEC] Completed SPI flash dump to 'bios_backup.bin'
[CHIPSEC] Time elapsed 31.595
(venv) ➔ chipsec file bios_backup.bin
bios_backup.bin: Intel serial flash for PCH ROM
```

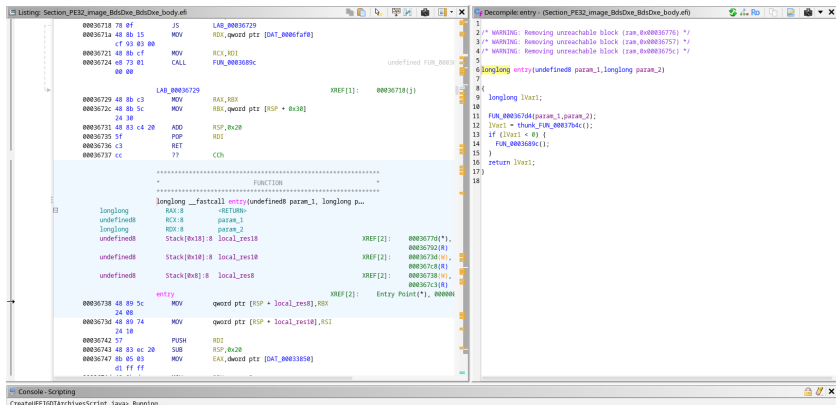
شکل ۱۱: خروجی دستور `chipsec_util spi dump`. نوع فایل باینری نیز در تصویر مشخص است.

این فایل به خودی یک فایل اجرایی نیست. برای دیدن اجزای آن باید از ابزاری مثل **UEFITool** [۵] استفاده کنیم. در این ابزار می‌توان برای مثال بخش‌های مربوط به SecureBoot را جست‌وجو کرد. فایل‌های PE (Portable Executable) فایل‌های اجرایی هستند که می‌توان در ghidra آن‌ها را decompile کرد.



شکل ۱۲: بخش BdsDxe در نتایج جست‌وجو برای SecureBoot آمده است.

حال می‌توان BdsDxe را Export Body کرد. خروجی آن فایل EFI است که ghidra آن را به عنوان فایل اجرایی می‌شناسد.



شکل ۱۳: فایل اجرایی در ghidra با موفقیت decompile شده است.

در مراحل بعدی برای مطالعه بیشتر فایل‌ها می‌توان از اسکریپت‌های آماده ghidra مثل **UEFI Surveyor** [۶] استفاده کرد. این اسکریپت‌ها داده‌ساختارهای مهم UEFI (شامل gBS و gRT) را مشخص می‌کنند که در فرآیند دیباگ ضروری هستند.

۷.۲ نحوه خوانده شدن متغیرهای EFI

از آن جا که آسیب‌پذیری لنو از نوع متغیرهای EFI با دسترسی اشتباه بود، لازم است ببینیم که چطور می‌توان متغیرهای EFI را در CHIPSEC خواند. در این بخش می‌بینیم که چه API ای از CHIPSEC برای این کار قرار داده شده و در لایه‌های پایین‌تر (HAL و OS Helper) چطور پیاده‌سازی شده است. ماژول‌هایی در CHIPSEC نوشته شده‌اند که متغیرهای EFI را بررسی می‌کنند، مثل `uefi.access_ufispec` که بررسی می‌کند آیا متغیرها دسترسی‌های درست طبق UEFI Specification دارند یا خیر. این ماژول در `chipsec/modules/common/uefi/access_ufispec.py` پیاده‌سازی شده است. در ابتدای تابع اجرای ماژول متغیرهای EFI خوانده شده‌اند.

```
def check_vars(self, do_modify: bool, attribute: Optional[int] = None) -> int:
    res = ModuleResult.PASSED
    vars = self._uefi.list_EFI_variables()
    if vars is None:
        self.logger.log_warning('Could not enumerate UEFI Variables from runtime.')
        self.logger.log_important("Note that UEFI variables may still exist, OS just did not expose runtime UEFI Variable API to read them.\nYou can extract variables directly from ROM file via 'chipsec_util.py uefi nvram bios.bin' command and verify their attributes manually.")
        self.result.setStatusBit(self.result.status.VERIFY)
    return ModuleResult.WARNING
```

شکل ۱۴: تابع اجرای ماژول `access_ufispec` از `self._uefi` متغیرها گرفته شده است.

```
class access_uefispec(BaseModule):
    def __init__(self):
        BaseModule.__init__(self)
        self._uefi = UEFI(self.cs)
```

شکل ۱۵: مقدار self._uefi برابر UEFI(self.cs) است. در ادامه می‌بینیم که UEFI دستورات سطح بالا را فراهم می‌کند و cs شی Chipset است که دستورات مرتبط با HAL (لایه زیرین معماری CHIPSEC) را در اختیار ما قرار می‌دهد.

```
def list_EFI_variables(self) -> Optional[Dict[str, List[Tuple[int, bytes, int, bytes, str, int]]]]:
    varlist = self.helper.list_EFI_variables()
    return varlist
```

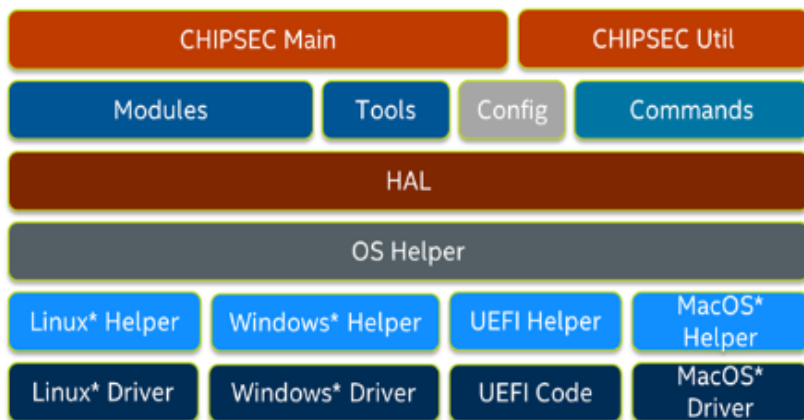
شکل ۱۶: تابع لیست کردن متغیرها در کلاس UEFI. صرفاً همان تابع در OS Helper را صدا می‌زند.

```
def kern_list_EFI_variables(self) -> Optional[Dict[str, List['EfiVariableType']]]:
    varlist = []
    off = 0
    hdr = 0
    attr = 0
    try:
        if os.path.isdir('/sys/firmware/efi/efivars'):
            varlist = os.listdir('/sys/firmware/efi/efivars')
        elif os.path.isdir('/sys/firmware/efi/vars'):
            varlist = os.listdir('/sys/firmware/efi/vars')
        else:
            return None
    except Exception:
        if logger().DEBUG:
            logger().log_error('Failed to read /sys/firmware/efi/[vars|efivars]. Folder does not exist')
        return None
    variables = dict()
    for v in varlist:
        name = v[:-37]
        guid = v[len(name) + 1:]
        if name and name is not None:
            variables[name] = []
            var = self.kern_get_EFI_variable_full(name, guid)
            (off, buf, hdr, data, guid, attr) = var
            variables[name].append(var)
    return variables
```

شکل ۱۷: در کلاس linuxhelper متغیرهای EFI از مسیر /sys/firmware/efi/efivars خوانده شده‌اند. این مسیر یک دایرکتوری است و به ازای هر متغیر فایل‌ی متناظر آن وجود دارد.

همانطور که می‌بینیم لایه‌های مختلف معماری CHIPSEC در این بخش مشخص شده‌اند. اصلی‌ترین لایه HAL است که سیستم عامل و firmware سیستم را abstract می‌کند و دستورات سطح بالا در اختیار کلاس‌ها

و ماژول‌ها قرار می‌دهد. لایه OS Helper پایین‌تر از HAL قرار دارد. برای مثال درایور لینوکس CHIPSEC که به آن در ابتدای پروژه برخوردیم در همین کلاس لود می‌شود.



CHIPSEC Architecture

شکل ۱۸: شکل کلی معماری CHIPSEC، از manual پروژه

۸.۲ متغیرهای تحت تاثیر آسیب‌پذیری لنوو

در این قسمت اطلاعات فنی دقیق‌تری از آسیب‌پذیری CVE-2022-3432 و آسیب‌پذیری‌های مرتبط به آن را جمع‌آوری کرده‌ایم. در تمام این آسیب‌پذیری‌ها، ساختن یا تغییر دادن یکی از متغیرهای EFI باعث می‌شود که Secure Boot غیرفعال شود. با غیرفعال شدن Secure Boot مهاجم می‌تواند بوت‌لودر را دستکاری کند، حمله‌ای که می‌تواند بعد از بوت شدن منجر به حملات دیگری در سیستم شود.

از منابع زیر جزئیات این آسیب‌پذیری‌ها جمع‌آوری شده است. در کل ۶ آسیب‌پذیری مرتبط روی لپ‌تاپ‌های Lenovo وجود دارد که ۵ تا از آن‌ها از همین نوع (ساخت متغیر NVRAM برای غیرفعال کردن Secure Boot) هستند.

- [ESET Research: When "secure" isn't secure at all \[3\]](#)
- [ESET Research Tweet \[7\]](#)
- [Lenovo Security Advisory: LEN-73440 \[8\]](#)
- [Lenovo Security Advisory: LEN-94952 \[2\]](#)

در تمام آسیب‌پذیری‌ها متغیرها از نوع Non-volatile هستند به طوری که در یک حمله متغیرها ساخته شده و سپس در reboot بعدی وضعیت Secure Boot را تغییر می‌دهند.

CVE-2021-3971 ۱.۸.۲

دراپورهای SecureBackDoorPeim و SecureBackDoor در صورت وجود متغیر زیر، Secure Boot را غیرفعال می‌کنند.

- Name: cE!
- Namespace GUID: LENOVO_BACKDOOR_NAMESPACE_GUID
- Attributes: NV + BS + RT (0x00000007)
- Value: Any non-null byte

CVE-2021-3972 ۲.۸.۲

دراپور ChgBootDxeHook با دیدن متغیر زیر Secure Boot را غیرفعال می‌کند.

- Name: ChgBootSecureBootDisable or ChgBootChangeLegacy
- Namespace GUID: LENOVO_BACKDOOR_NAMESPACE_GUID
- Attributes: NV + BS + RT (0x00000007)
- Value: Any non-null byte

But still, if there are some readers interested in how it can be done by invoking the SW SMI handler, here is the [CHIPSEC](#) command that can be used to create the ChgBootSecureBootDisable variable:

```
chipsec_util.py smi send 0 0xCA 0x53 0x53CA 0x1D 0 0xB2
```

شکل ۱۹: تعریف متغیر بالا در CHIPSEC که در پست ESET آمده است.

CVE-2022-3430 ۳.۸.۲

دراپور WmiSetupUnderOsDxe متغیر L05WSBD را می‌خواند و اگر مقدار L05SecureBootData.Action (یک field از این متغیر) برابر ۲ باشد، Secure Boot غیرفعال می‌شود.

دراپور BootOrderDxe در صورت وجود متغیر BootOrderSecureBootDisable یا BootOrderDualBootMode، وضعیت Secure Boot را غیرفعال می‌کند.

دراپور BdsDxe متغیر L05SecBootSmm را می‌خواند. اگر مقدار آن صفر باشد Secure Boot را غیرفعال می‌کند.

۹.۲ توسعه ماژول غیرفعال برای تشخیص متغیرهای درپشتی

با توجه به بررسی‌های انجام‌شده روی معماری CHIPSEC و ویژگی‌های فنی آسیب‌پذیری‌های ذکر شده، استراتژی اول یعنی غربالگری غیرفعال (Passive Scan) برای تشخیص خانواده آسیب‌پذیری‌های مرتبط با متغیرهای درپشتی پیاده‌سازی شد. ماژول پیاده‌سازی شده در مسیر `sut/check_backdoor_vars.py` قرار دارد. منطق پیاده‌سازی این ماژول مشابه ماژول نمونه است که در آن وجود متغیرها و مقدارشان بررسی می‌شود. ماژول طراحی شده با بهره‌گیری از لایه انتزاع سخت‌افزاری CHIPSEC HAL، متغیرهای محیط اجرای سیستم عامل (مسیر `/sys/firmware/efi/efivars`) را فراخوانی کرده و با لیست آسیب‌پذیری‌ها تطبیق می‌دهد. ماژول سفارشی توسعه داده شده با موفقیت روی سخت‌افزار واقعی سیستم آزمایشگاهی اجرا گردید. همان‌طور که در تصویر خروجی مشخص است، دراپور اختصاصی چارچوب CHIPSEC بارگذاری شده و دسترسی لازم به حافظه پایداری سیستم جهت بررسی متغیرها برقرار شده است. پس از اسکن و تحلیل کامل متغیرهای موجود، هیچ‌گونه نشانه‌ای از متغیرهای آسیب‌پذیر مرتبط با بک‌دورهای لنوو یافت نشد و عملکرد صحیح ماژول با گزارش وضعیت نهایی PASSED مورد تأیید قرار گرفت.

```
[CHIPSEC] OS      : Linux 7.0.11-200.fc44.x86_64 #1 SMP PREEMPT_DYNAMIC Mon Jun 12 22:50:37 UTC 2026 x86_64
[CHIPSEC] Python  : 3.14.5 (64-bit) - Enabled GIL
[CHIPSEC] Helper   : LinuxHelper (/home/parsa/University/aziot/chipsec/chipsec/helper/linux/chipsec.ko)
[CHIPSEC] Platform: Mobile 6th Generation Core Processor Quad Core (Skylake H)
[CHIPSEC] CPUID    : 506E3
[CHIPSEC] VID      : 8086
[CHIPSEC] DID      : 1910
[CHIPSEC] RID      : 07
[CHIPSEC] PCH      : Intel HM170 (100 series) PCH
[CHIPSEC] VID      : 8086
[CHIPSEC] DID      : A14E
[CHIPSEC] RID      : 31

[+] loaded chipsec.modules.sut.check_backdoor_vars
[*] running loaded modules ..

[*] Running module: chipsec.modules.sut.check_backdoor_vars
[#] INFORMATION: Starting passive compliance scan for Secure Boot vulnerabilities...
[+] No known firmware backdoor variables found in NVRAM. System looks clean!
```

شکل ۲۰: خروجی اجرای ماژول غیرفعال

۱۰.۲ بررسی CVE-2022-3432 بر روی دستگاه

در آسیب‌پذیری CVE-2022-3432 چنانچه متغیر EFI زیر تعریف شده باشد و مقدار آن صفر باشد، Secure Boot غیرفعال می‌شود.

- Name: L05SecBootSmm
- GUID: CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF

برای بررسی این آسیب‌پذیری، ابتدا این متغیر را اضافه می‌کنیم و مقدار آن را صفر قرار می‌دهیم. سپس Secure Boot را فعال می‌کنیم و متغیر را بررسی می‌کنیم. خواهیم دید که مقدار متغیر صفر است و Secure Boot فعال است. در نتیجه آسیب‌پذیری شامل این دستگاه نشده است. در طول این فرآیند برای اجرای ماژول غیرفعال و همچنین استفاده از chipsec_util، مراحل اضافه کردن Machine Owner Key و امضای درایور chipsec را نیز توضیح خواهیم داد.

۱.۱۰.۲ افزودن متغیر

```
(venv) + chipsec sudo python3 chipsec_util.py uefi var-read L05SecBootSmm CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF (3.88s) git:(out-iotlab) 18:35

#####
##                               ##
## CHIPSEC: Platform Hardware Security Assessment Framework ##
##                               ##
#####
[CHIPSEC] Version : 1.13.20
[CHIPSEC] Arguments: uefi var-read L05SecBootSmm CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF

[!] Results from this system may be incorrect.
WARNING: Not loading configurations. Platform will remain unknown.

[CHIPSEC] OS      : Linux 7.0.12-201.fc44.x86_64 #1 SMP PREEMPT_DYNAMIC Thu Jun 11 01:30:16 UTC 2026 x86_64
[CHIPSEC] Python  : 3.14.5 (64-bit) - Enabled GIL
[CHIPSEC] Helper   : LinuxHelper (/home/parsa/University/aziot/chipsec/chipsec/helper/linux/chipsec.ko)
[CHIPSEC] Platform: Unrecognized Platform
[CHIPSEC] CPUID    : FFFF
[CHIPSEC] VID      : FFFF
[CHIPSEC] DID      : FFFF
[CHIPSEC] RID      : FF
[CHIPSEC] Executing command 'uefi' with args ['var-read', 'L05SecBootSmm', 'CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF']

[CHIPSEC] Reading EFI variable Name='L05SecBootSmm' GUID='{CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF}' to 'None' via Variable API..
[CHIPSEC] Time elapsed 0.001
```

شکل ۲۱: با دستور uefi var-read chipsec_util متغیر را خوانده‌ایم. قبل از این فرآیند متغیر L05SecBootSmm تعریف نشده است.

```
(venv) ➔ chipsec xxd set_0.bin
00000000: 00
(venv) ➔ chipsec sudo python3 chipsec_util.py uefi var-write L05SecBootSmm CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF set_0.bin

#####
## CHIPSEC: Platform Hardware Security Assessment Framework ##
##
#####
[CHIPSEC] Version : 1.13.20
[CHIPSEC] Arguments: uefi var-write L05SecBootSmm CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF set_0.bin

[!] Results from this system may be incorrect.
WARNING: Not loading configurations. Platform will remain unknown.

[CHIPSEC] OS : Linux 7.0.12-201.fc44.x86_64 #1 SMP PREEMPT_DYNAMIC Thu Jun 11 01:30:16 UTC 2026 x86_64
[CHIPSEC] Python : 3.14.5 (64-bit) - Enabled GIL
[CHIPSEC] Helper : LinuxHelper (/home/parsa/University/aziot/chipsec/chipsec/helper/linux/chipsec.ko)
[CHIPSEC] Platform: Unrecognized Platform
[CHIPSEC] CPUID: FFFF
[CHIPSEC] VID: FFFF
[CHIPSEC] DID: FFFF
[CHIPSEC] RID: FF
[CHIPSEC] Executing command 'uefi' with args ['var-write', 'L05SecBootSmm', 'CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF', 'set_0.bin']

[CHIPSEC] writing EFI variable Name='L05SecBootSmm' GUID=(CE5E5928-C7A3-4602-AD9E-C9DAF94EBFCF) from 'set_0.bin' via Variable API..
[CHIPSEC] status: EFI_SUCCESS
[CHIPSEC] writing EFI variable was successful
[CHIPSEC] Time elapsed 0.075
```

شکل ۲۲: یک فایل باینری حاوی بایت صفر می‌سازیم و با دستور `uefi var-write` آن را در متغیر `L05SecBootSmm` می‌نویسیم.

```
778 -----
779 EFI Variable (offset = 0x0):
780 -----
781 Name      : L05SecBootSmm
782 Guid      : ce5e5928-c7a3-4602-ad9e-c9daf94ebfcf
783 Attributes: 0x7 ( NV+BS+RT )
784 Data:
785 00
```

شکل ۲۳: بدون فعال کردن Secure Boot ریویز می‌کنیم و با دستور `uefi var-list` می‌بینیم که متغیر `L05SecBootSmm` با مقدار داده شده وجود دارد.

```

→ efivars pwd
/sys/firmware/efi/efivars
→ efivars xxd L05SecBootSmm-ce5e5928-c7a3-4602-ad9e-c9daf94ebfcf
00000000: 0700 0000 00 .....
→ efivars mokutil --sb-state
SecureBoot enabled
→ efivars █

```

شکل ۲۴: در ری بوت بعدی Secure Boot را فعال می‌کنیم و دستی متغیر را در `/sys/firmware/efi/efivars` می‌خوانیم. به ازای هر متغیر یک فایل تعریف شده است. در این فایل ۴ بایت اول attribute ها هستند (NV+BS+RT). ادامه فایل محتوای متغیر است. با خروجی دستور `mokutil` می‌توان دید که Secure Boot فعال است.

می‌توان دید که متغیر صفر است اما Secure Boot غیرفعال نشده است. به همین خاطر آسیب‌پذیری شامل این دستگاه نشده است. در گزارش CVE می‌توان دید که این آسیب‌پذیری بر روی مدل 14ISK گزارش داده شده است و ما آن را در 15ISK بررسی کرده‌ایم. به احتمال زیاد آسیب‌پذیری تنها محدود به همان نسخه باشد.

۲.۱۰.۲ اجرای ماژول غیرفعال با Secure Boot

در ادامه مراحل تولید `Machine Owner Key` برای امضای درایور `chipsec` را توضیح خواهیم داد. با اضافه کردن کلید ساخته شده به سیستم، درایور `chipsec` می‌تواند در محیط Secure Boot بدون خطا بارگذاری شود. به دو منظور این کار را انجام می‌دهیم. اول اینکه ماژول‌های پیش‌فرض و ماژول غیرفعال را اجرا می‌کنیم. خواهیم دید که ماژول‌های بیشتری از `chipsec` پذیرفته می‌شوند، اما ماژول غیرفعال به خاطر وجود متغیر درپشتی خطا خواهد داد. دوم اینکه با داشتن چنین محیطی و با در نظر داشتن آسیب‌پذیری‌های Secure Boot، در صورت نیاز محیط آماده برای تست ماژول‌ها حتی با Secure Boot روشن خواهیم داشت.

```

root@lenovoy700:~/mok# openssl req -new -x509 -newkey rsa:2048 -nodes \
-keyout mok.key -out mok.der -outform DER \
-days 365

```

شکل ۲۵: کلید MOK را می‌توان با `openssl` ساخت. این کلید از نوع RSA است و خروجی آن کلید خصوصی با پسوند `key` و کلید عمومی با پسوند `der` است.

برای امضای درایورها اسکریپت `sign-file` در بسته `kernel-devel` در فلدرها تعریف شده است. این اسکریپت کلید خصوصی، عمومی و درایور با پسوند `ko` را ورودی می‌گیرد و درایور را امضا می‌کند.

```
(venv) ➔ chipsec modinfo drivers/linux/chipsec.ko
filename:      /home/parsa/University/aziot/chipsec/drivers/linux/chipsec.ko
license:       GPL
depends:
name:          chipsec
retpoline:     Y
vermagic:      7.0.12-201.fc44.x86_64 SMP preempt mod_unload
sig_id:        PKCS#7
signer:        lenovoy700.lan
sig_key:       3B:8E:38:87:AD:F6:DB:9A:5B:D0:FD:D7:B9:48:85:61:F2:E9:8A:DC
sig_hashalgo:  sha256
signature:     60:85:E3:68:15:C5:70:82:0A:3B:81:45:EA:40:60:FE:3C:9F:6A:94:
95:4D:D0:67:EC:47:8E:7B:C4:FA:C9:5C:F7:45:30:55:F4:6D:9E:1B:
46:36:EA:2D:72:AF:DC:54:05:EC:0C:D9:ED:F2:7B:78:CD:D5:A1:2F:
20:7B:69:6C:8F:20:3D:CF:D9:EA:29:DD:19:32:3B:E7:09:BA:F3:9A:
C9:45:7A:62:3B:91:81:13:20:D4:32:CD:6A:6D:88:78:59:93:A2:0D:
AD:85:4D:34:99:37:7C:32:74:30:67:A4:5E:76:F9:06:E7:61:FD:FF:
E1:00:70:4D:19:06:FE:A6:31:1A:9C:3C:49:4C:E9:84:05:93:A6:58:
B3:B6:ED:74:58:DD:58:17:DE:F4:44:60:66:76:22:81:33:7B:47:F9:
91:FC:AC:6E:BB:47:A0:D6:1C:E3:C9:75:40:10:07:41:9E:B8:A6:48:
B2:5F:6C:18:17:04:04:73:38:40:64:DF:43:61:DB:59:E8:33:41:C4:
05:1A:8B:F3:87:BF:A3:49:C0:C7:36:7D:30:8C:D9:B2:CD:66:EE:9B:
22:CE:59:48:3B:EA:BA:4A:1C:13:B4:67:6D:AD:AE:B3:F8:9E:9F:0F:
B4:18:F8:E9:55:31:16:6F:9F:DA:18:5C:E2:42:C7:CA
parm:          a1:ulong
parm:          a2:ulong
```

شکل ۲۶: با دستور modinfo می‌توان دید که درایور امضا شده است.

برای تایید کردن کلید ساخته شده دستور `mokutil -import mok.der` را اجرا می‌کنیم. در ریوت بعدی می‌توانیم کلید MOK را اضافه کنیم. پس از ریوت `chipsec__main` بدون خطا اجرا می‌شود.

```
[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed          0.221
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed       16:
[+] PASSED: chipsec.modules.common.bios_kbrd_buffer
[+] PASSED: chipsec.modules.common.bios_smi
[+] PASSED: chipsec.modules.common.bios_ts
[+] PASSED: chipsec.modules.common.ia32cfg
[+] PASSED: chipsec.modules.common.memconfig
[+] PASSED: chipsec.modules.common.memlock
[+] PASSED: chipsec.modules.common.remap
[+] PASSED: chipsec.modules.common.rtclock
[+] PASSED: chipsec.modules.common.secureboot.variables
[+] PASSED: chipsec.modules.common.smm
[+] PASSED: chipsec.modules.common.smm_code_chk
[+] PASSED: chipsec.modules.common.smm_dma
[+] PASSED: chipsec.modules.common.smrr
[+] PASSED: chipsec.modules.common.spd_wd
[+] PASSED: chipsec.modules.common.spi_fdopss
[+] PASSED: chipsec.modules.common.uefi.access_uefispec
[CHIPSEC] Modules information 1:
[#] INFORMATION: chipsec.modules.common.cpu.cpu_info
[CHIPSEC] Modules failed      5:
[-] FAILED: chipsec.modules.common.bios_wp
[-] FAILED: chipsec.modules.common.me_mfg_mode
[-] FAILED: chipsec.modules.common.spi_access
[-] FAILED: chipsec.modules.common.spi_desc
[-] FAILED: chipsec.modules.common.spi_lock
[CHIPSEC] Modules with warnings 3:
WARNING: chipsec.modules.common.cpu.spectre_v2
WARNING: chipsec.modules.common.debugenabled
WARNING: chipsec.modules.common.sgx_check
```

شکل ۲۷: خروجی chipsec_main وقتی Secure Boot فعال است. چند مازولی که قبلاً warning داشتند اکنون پذیرفته شده‌اند.

```
[CHIPSEC] Reading EFI variable Name='L05SecBootSmm' GUID={CE5E928-C7A3-4602-AD9E-C9DAF94EBFCF} to 'None' via Variable API..
[uefi] EFI variable CE5E928-C7A3-4602-AD9E-C9DAF94EBFCF:L05SecBootSmm :
00 |
[CHIPSEC] Time elapsed 0.001
(venv) > chipsec mokutil --sb-state
SecureBoot enabled
(venv) > chipsec
```

شکل ۲۸: مازول غیرفعال نیز در این اجرا خطا می‌دهد. زیرا متغیر L05SecBootSmm تعریف شده و مقدار آن صفر است.

۱۱.۲ اجرای chipsec در محیط OVMF

پروژه OVMF بخشی از پروژه EDK II است که امکان اجرای برنامه‌های UEFI در ماشین مجازی را فراهم می‌کند. در این قسمت سعی می‌کنیم chipsec را در OVMF اجرا کنیم. با داشتن چنین محیطی می‌توان ماژول‌هایی مختص به OVMF نوشت و راحت‌تر و سریع‌تر ماژول‌ها را تست کرد.

در پروژه chipsec اسکریپت `qemu_efi.py` قرار داده شده است که مراحل ساخت image و اجرای آن با `qemu` را انجام می‌دهد. با کمک این اسکریپت پس از بوت شدن ماشین مجازی، وارد UEFI Shell خواهیم شد و در آنجا به پایتون و ماژول‌ها و ابزارهای chipsec دسترسی خواهیم داشت.

```
# Unzip Python into the temporary directory
print("Extracting {} to {} ...".format(PYTHON_INSTALL_ZIP.name, tmpdir))
with zipfile.ZipFile(PYTHON_INSTALL_ZIP) as python_zip:
    python_zip.extractall(tmpdir)

# Copy Chipsec
print("Copying chipsec to {} ...".format(tmpdir))
for extension in (".py", ".xml", ".xsd", ".ini", ".json", ".txt"):
    for filepath in CHIPSEC_BASE.glob("**/*" + extension):
        destination = Path(tmpdir) / "chipsec" / filepath.relative_to(CHIPSEC_BASE)
        destination.parent.mkdir(parents=True, exist_ok=True)
        shutil.copy(filepath, destination)

# Create startup.nsh
if startup_nsh:
    with (Path(tmpdir) / "startup.nsh").open("w") as f:
        f.write(startup_nsh)

# Convert to QCow image
print("Converting to QCow2 format in {} ...".format(filename))
subprocess.run(
    ("qemu-img", "convert", "-c", "-f", "vfat", "-O", "qcow2", "fat:" + tmpdir, filename),
    check=True
)
```

شکل ۲۹: ساخت ایمیج در اسکریپت `qemu_efi.py`

```

qemu_cmd = [
    "qemu-system-x86_64",
    # Make OVMF boot directly in UEFI shell instead of trying network boot (PXE)
    "-nic", "none",
    # Enable exiting by entering "reset" in UEFI shell
    "-no-reboot",
    # Start OVMF
    "-drive", "if=pflash,format=raw,readonly=on,file=" + ovmf_path,
]
if not args.gui:
    qemu_cmd += ["-nographic"]

if args.memory:
    qemu_cmd += ["-m", args.memory]

if args.host_cpu:
    qemu_cmd += ["-cpu", "host", "-enable-kvm"]

if args.machine:
    qemu_cmd += ["-machine", args.machine]

with tempfile.NamedTemporaryFile(prefix="efidisk_") as tmpdisk:
    create_efi_disk(tmpdisk.name, startup_nsh)
    qemu_cmd += ["-drive", "format=qcow2,file=" + tmpdisk.name]

    if args.qemu_arg:
        qemu_cmd += [arg[1:] if arg.startswith(" ") else arg for arg in args.qemu_arg]

    print("Launching {}".format(" ".join(qemu_cmd)))
    subprocess.run(qemu_cmd, check=True)

```

شکل ۳۰: اجرای qemu با OVMF و ایمپج حاوی chipsec

در اجرای اولیه اسکریپت به خطا می‌خوریم که نشان می‌دهد بخشی از فایل‌های chipsec به درستی منتقل نشده‌اند. لازم است تغییر زیر را روی `qemu_img.py` اعمال کنیم تا بدون خطا اجرا شود.

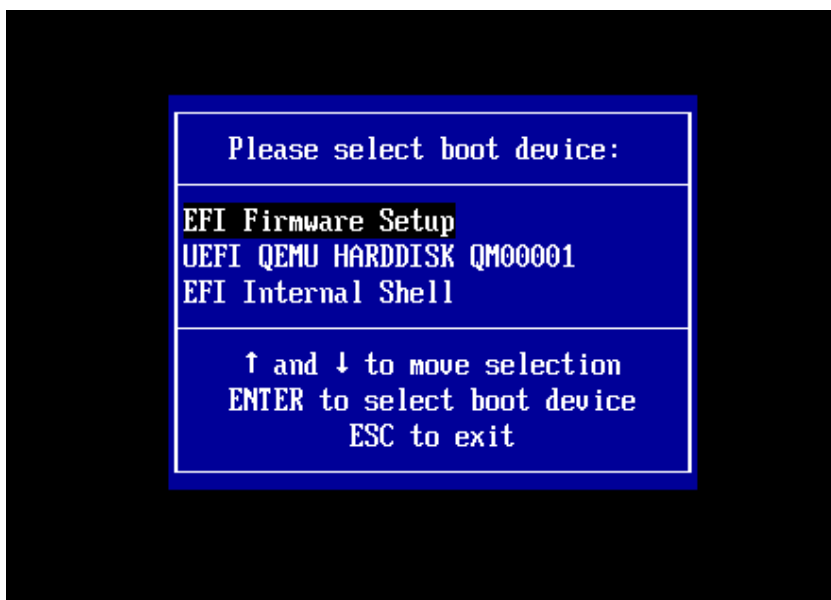
```

--- a/scripts/qemu_efi.py
+++ b/scripts/qemu_efi.py
@@ -76,7 +76,7 @@ def create_efi_disk(filename, startup_nsh=None):

    # Copy Chipsec
    print("Copying chipsec to {}".format(tmpdir))
    for extension in (".py", ".xml", ".xsd"):
+   for extension in (".py", ".xml", ".xsd", ".ini", ".json", ".txt"):
        for filepath in CHIPSEC_BASE.glob("**/*" + extension):
            destination = Path(tmpdir) / "chipsec" / filepath.relative_to(CHIPSEC_BASE)
            destination.parent.mkdir(parents=True, exist_ok=True)

```

شکل ۳۱: تغییرات اسکریپت `qemu_img.py`



شکل ۳۲: پس از اجرای اسکریپت محیط ماشین مجازی را خواهیم دید. در این مرحله لازم است وارد EFI Internal Shell شویم.

به طور پیش فرض اسکریپت startup.nsh صدا زده می شود که پس از یک دور اجرای chipsec_main از شل خارج می شود. همچنین می توان از اجرای startup.nsh صرف نظر کرد که در این حالت مستقیماً وارد UEFI Shell خواهیم شد.


```

FS0:\chipsec> ls
Directory of: FS0:\chipsec\
06/16/2026  21:20 <DIR>          8,192 .
06/16/2026  21:20 <DIR>           0 ..
06/16/2026  21:20 <DIR>          8,192 .vscode
06/16/2026  21:20 <DIR>          8,192 chipsec
06/16/2026  21:20             21,945 chipsec_main.py
06/16/2026  21:20 <DIR>          8,192 chipsec_tools
06/16/2026  21:20             9,078 chipsec_util.py
06/16/2026  21:20 <DIR>          8,192 docs
06/16/2026  21:20 <DIR>          8,192 drivers
06/16/2026  21:20             54 linux_requirements.txt
06/16/2026  21:20 <DIR>          8,192 scripts
06/16/2026  21:20            10,727 setup.py
06/16/2026  21:20 <DIR>          8,192 tests
06/16/2026  21:20 <DIR>          8,192 venv
06/16/2026  21:20             64 windows_requirements.txt
          5 File(s)          41,868 bytes
         10 Dir(s)
FS0:\chipsec> python.efi chipsec_main.py --help

#####
##                                     ##
##  CHIPSEC: Platform Hardware Security Assessment Framework  ##
##                                     ##
#####
[CHIPSEC] Version :
[CHIPSEC] Arguments: --help

usage: chipsec_main.py [options]

```

شکل ۳۳: اجرای chipsec به صورت دستی در شل

```
[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed          28.000
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed       3:
[+] PASSED: chipsec.modules.common.bios_kbrd_buffer
[+] PASSED: chipsec.modules.common.bios_smi
[+] PASSED: chipsec.modules.common.uefi.access_uefispec
[CHIPSEC] Modules information  1:
[#] INFORMATION: chipsec.modules.common.cpu.cpu_info
[CHIPSEC] Modules failed      4:
[-] FAILED: chipsec.modules.common.cpu.spectre_v2
[-] FAILED: chipsec.modules.common.ia32cfg
[-] FAILED: chipsec.modules.common.memlock
[-] FAILED: chipsec.modules.common.smrr
[CHIPSEC] Modules with warnings 8:
WARNING: chipsec.modules.common.bios_ts
WARNING: chipsec.modules.common.bios_wp
WARNING: chipsec.modules.common.secureboot.variables
WARNING: chipsec.modules.common.smm_dma
WARNING: chipsec.modules.common.spi_access
WARNING: chipsec.modules.common.spi_desc
WARNING: chipsec.modules.common.spi_fdopss
WARNING: chipsec.modules.common.spi_lock
[CHIPSEC] Modules not applicable 15:
NOT APPLICABLE: chipsec.modules.common.cet
NOT APPLICABLE: chipsec.modules.common.cpu.ia_untrusted
NOT APPLICABLE: chipsec.modules.common.debugenabled
NOT APPLICABLE: chipsec.modules.common.me_mfg_mode
NOT APPLICABLE: chipsec.modules.common.memconfig
NOT APPLICABLE: chipsec.modules.common.remap
NOT APPLICABLE: chipsec.modules.common.rom_armor
NOT APPLICABLE: chipsec.modules.common.rtclock
NOT APPLICABLE: chipsec.modules.common.sgx_check
NOT APPLICABLE: chipsec.modules.common.smm
```

شکل ۳۴: نمونه اجرای chipsec_main

۱۲.۲ آسیب‌پذیری CVE-2023-48733 مرتبط با OVMF

آسیب‌پذیری CVE-2023-48733 [۹] یک پیکربندی ناامن پیش‌فرض در بسته EDK2 دبین است که امکان دور زدن Secure Boot را از طریق UEFI Shell فراهم می‌کند. در ساخت دبین از EDK2، گزینه‌ای که اجرای UEFI Shell را مجاز می‌شمارد به صورت پیش‌فرض فعال بوده است. در نتیجه حتی زمانی که Secure Boot فعال است، مهاجمی که از درون سیستم عامل دسترسی دارد می‌تواند با انتخاب EFI Shell در منوی بوت، مکانیزم Secure Boot را دور بزند.

بر اساس اطلاعیه امنیتی دبین [۱۰]، وصله این آسیب‌پذیری با غیرفعال کردن UEFI Shell هنگامی که Secure Boot فعال است اعمال شده است. آخرین نسخه آسیب‌پذیر [۱۱] بسته edk2 در دبین نسخه -2022.08 است و اولین نسخه تصحیح‌شده [۱۲] نسخه deb12u2+2022.11-6 است.

۱۳.۲ نصب سیستم عامل مجازی با QEMU و OVMF

در قسمت‌های قبلی توانستیم CHIPSEC را به صورت شبیه‌سازی شده در OVMF اجرا کنیم. این قابلیت که در یک اسکریپت در خود پروژه CHIPSEC قرار داده شده بود، یک نسخه از محتوای پروژه به همراه python.efi را در یک دیسک QEMU آماده می‌کرد طوری که با بوت کردن EFI Shell دسترسی برای اجرای CHIPSEC وجود داشت.

در این آسیب‌پذیری OVMF، زمانی که Secure Boot فعال باشد انتظار داریم EFI Shell قابل بوت شدن نباشد. به همین خاطر باید CHIPSEC را در محیط OS اجرا کنیم و داخل آن با اجرا کردن CHIPSEC آسیب‌پذیری را بررسی کنیم. در گزارش توضیحات اجرای CHIPSEC در یک سیستم عامل با Secure Boot فعال را شرح دادیم. در اینجا جزئیات مربوط به شبیه‌سازی و نصب OS را توضیح می‌دهیم.

```
install-iso() {
    qemu-system-x86_64 \
    -machine q35,smm=on -accel kvm -cpu host \
    -m 5G -smp 2 \
    -global driver=cfi.pflash01,property=secure,value=on \
    -drive if=pflash,format=raw,unit=0,file=./new-debian/OVMF/OVMF_CODE_4M.secboot.fd,readonly=on \
    -drive if=pflash,format=raw,unit=1,file=./new-debian/my_VARS_4M.ms.fd \
    -global ICH9-LPC.disable_s3=1 \
    -drive file=./disk/debian.qcow2,if=virtio,format=qcow2 \
    -cdrom ./disk/debian-13.5.0-amd64-netinst.iso \
    -nic none \
    -net nic -net user
}
```

شکل ۳۵: دستور QEMU برای نصب OS

در اینجا از سیستم عامل دبین نسخه 13.5.0 استفاده می‌کنیم. دبین به خاطر استفاده از بوت‌لودر shim امضای مایکروسافت را دارد و می‌تواند با Secure Boot نیز بوت شود. فایل my_VARS_4M.ms.fd از روی فایل debian.qcow2 OVMF_4M.ms.fd کپی شده است که امضای مایکروسافت را در خود دارد. فایل debian.qcow2 دیسک خالی برای نصب دبین روی آن است. در این دستور ISO نصب OS بوت می‌شود و مراحل نصب انجام می‌شود.

```
vm-old() {
    qemu-system-x86_64 \
    -machine q35,smm=on -accel kvm -cpu host \
    -m 4G -smp 2 \
    -global driver=cfi.pflash01,property=secure,value=on \
    -drive if=pflash,format=raw,unit=0,file=./old-debian/OVMF/OVMF_CODE_4M.secboot.fd,readonly=on \
    -drive if=pflash,format=raw,unit=1,file=./new-debian/my_VARS_4M.ms.fd \
    -global ICH9-LPC.disable_s3=1 \
    -drive file=./disk/debian.qcow2,if=virtio,format=qcow2 \
    -nic none \
    -net nic -net user
}
```

شکل ۳۶: دستور QEMU برای اجرای OS با OVMF آسیب‌پذیر

برای اجرا دیگر نیازی به فایل ISO نصب نیست. همچنین با تغییر pflash می‌توان انتخاب کرد که کدام نسخه از OVMF استفاده شود. برای اینکه متغیرهای EFI در تمام اجراها یکسان باشند از یک فایل

my_VARS_4M.ms.fd استفاده شده است.

پس از پایان نصب لازم است پروژه CHIPSEC برای اجرا آماده شود. از آنجا که Secure Boot فعال است باید مراحل تولید (MOK) Machine Owner Key، امضای chipsec.ko با MOK، و در آخر وارد کردن MOK انجام شود. توضیحات مربوط به این قسمت در گزارش پنجم آمده است.

۱۴.۲ تلاش اول: جست‌وجوی GUID در حافظه فلش

در نسخه آسیب‌پذیر 2022.08-1 یک FullShell در حافظه فلش وجود دارد که در آخرین نسخه 2025.11-4 نیست. به همین خاطر در تلاش اول برای بررسی آسیب‌پذیری در یک ماژول CHIPSEC، ابتدا مطمئن می‌شویم که Secure Boot فعال باشد و سپس بررسی می‌کنیم که GUID مرتبط با FullShell در حافظه فلش آمده باشد. اگر FullShell ظاهر نشده باشد PASS و در غیر این صورت FAIL است.

```
if not secureboot_enabled:
    self.logger.log_warning("Secure Boot is not enabled. skipping shell presence check")
    return ModuleResult.WARNING

FLASH_BASE = 0xFFC00000 # 4M build; OVMF_CODE_4M.secboot.fd
FLASH_SIZE = 0x400000
fw = bytes(self.cs.mem.read_physical_mem(FLASH_BASE, FLASH_SIZE))

shell_present = uuid.UUID(FULLSHELL_GUID).bytes_le in fw

if secureboot_enabled and shell_present:
    self.logger.log_failed("Secure Boot is enabled but UEFI Full Shell (FULLSHELL_GUID) is present in firmware. shell can be used to bypass Secure Boot")
    return ModuleResult.FAILED
else:
    self.logger.log_passed("Secure Boot is enabled and no unauthorized UEFI Full Shell was found in firmware")
    return ModuleResult.PASSED
```

شکل ۳۷: خواندن حافظه و بررسی وجود FULLSHELL_GUID

با اجرای این ماژول بر روی نسخه‌های متفاوت OVMF متوجه می‌شویم که این روش برای بررسی آسیب‌پذیری مناسب نیست. در آخرین نسخه آسیب‌پذیر 2022.08-1 یک FullShell با GUID داده شده وجود دارد و ماژول به درستی FAIL می‌شود. ولی در اولین نسخه تصحیح شده 2022.11-6+deb12u2 هم همچنان FullShell وجود دارد و تنها تفاوت آن با نسخه قبلی این است که نمی‌توان آن را بوت کرد (با انتخاب دستی EFI Shell در UEFI Firmware Settings). در آخرین نسخه 2025.11-4 دیگر FullShell وجود ندارد اما همچنان GUID آن در حافظه وجود دارد (با کمک uefifool می‌بینیم که GUID در بدنه BdsDxe آمده است). پس هر دو نسخه جدید نیز FAIL می‌شوند. با چنین ماژولی نمی‌توان وجود آسیب‌پذیری را بررسی کرد.

۱۵.۲ تلاش دوم: بررسی قابل بوت بودن شل

در تلاش دوم سعی می‌کنیم شل را مستقیماً بوت کنیم. به این شکل که با کمک دستور efibootmgr ابتدا به دنبال EFI Shell می‌گردیم و آن را به عنوان BootNext تنظیم می‌کنیم. سپس از کاربر می‌خواهیم که OS را ریboot کند.

برای اینکه بررسی شل اجرا شده است یا خیر، از ایده‌ای که در اولین اجرای CHIPSEC در OVMF دیدیم استفاده می‌کنیم: اگر فایل startup.nsh در پارتیشن EFI قرار دهیم، می‌دانیم که در صورت بوت شدن شل چنین فایل‌ی به صورت خودکار اجرا می‌شود.

در فایل startup.nsh یک متغیر برای خودمان به نام ShellMark را برابر 1 تنظیم می‌کنیم. همچنین در ابتدا این متغیر را صفر قرار می‌دهیم. اگر پس از ریوت مقدار متغیر عوض شده باشد یعنی آسیب‌پذیری وجود دارد. مزیت این روش این است که قابلیت بوت شدن شل در شرایط واقعی بررسی می‌کند و برای نسخه 2022.11-6+deb12u2 که شل دارد اما قابل بوت نیست هم به درستی تشخیص می‌دهد.

```
def setup_boot_to_shell(self):
    result = subprocess.run(['efibootmgr'], capture_output=True, text=True)
    if result.returncode != 0:
        self.logger.log_error(f"efibootmgr failed: {result.stderr.strip()}")
        return ModuleResult.ERROR

    shell_boot_num = None
    for line in result.stdout.splitlines():
        m = re.match(r'^Boot([0-9A-Fa-f]{4})\s*\s+EFI Internal Shell', line)
        if m:
            shell_boot_num = m.group(1)
            break

    if shell_boot_num is None:
        self.logger.log_good("No 'EFI Internal Shell' boot option found. System is not vulnerable.")
        return ModuleResult.PASSED

    self.logger.log(f"Found EFI Internal Shell boot option: Boot{shell_boot_num}")
```

شکل ۳۸: بررسی وجود EFI Shell بین گزینه‌های بوت

```
import os
if os.path.exists(STARTUP_NSH_PATH):
    self.logger.log_error(f"{STARTUP_NSH_PATH} already exists. Remove it before running setup.")
    return ModuleResult.ERROR

with open(STARTUP_NSH_PATH, 'w') as f:
    f.write(STARTUP_NSH_CONTENT)
self.logger.log(f"Wrote {STARTUP_NSH_PATH}")
```

شکل ۳۹: نوشتن فایل /boot/efi/startup.nsh

```
(venv) debian@debian:~/chipsec/chipsec$ sudo cat /boot/efi/startup.nsh
@echo -off
setvar ShellMark -guid CC80C59E-7482-11F1-AA45-507B9DBF9094 -nv -bs -rt =0x01
reset
(venv) debian@debian:~/chipsec/chipsec$ █
```

شکل ۴۰: محتویات فایل startup.nsh

```

self._uefi.set_EFI_variable('ShellMark', SHELLMARK_GUID, b'\x00', attrs=SHELLMARK_ATTRS)
self.logger.log("Set ShellMark EFI variable to 0x00")

nb_result = subprocess.run(['efibootmgr', '-n', shell_boot_num], capture_output=True,
text=True)
if nb_result.returncode != 0:
    self.logger.log_error(f"efibootmgr -n failed: {nb_result.stderr.strip()}")
    return ModuleResult.ERROR

self.logger.log(f"Next boot set to Boot{shell_boot_num} (EFI Internal Shell)")
self.logger.log("Please reboot now. After the system comes back up, run this module without
'setup' to check the result.")
return ModuleResult.WARNING

```

شکل ۴۱: ساختن متغیر ShellMark با مقدار اولیه صفر و سپس تعیین BootNext با دستور `efibootmgr -n`

```

def check_mark(self):
    data = self._uefi.get_EFI_variable(EFI_VAR_NAME_SecureBoot, EFI_GLOBAL_VARIABLE_GUID)
    secureboot_enabled = (data is not None) and (len(data) == 1) and (data[0] == 0x1)

    if not secureboot_enabled:
        self.logger.log_warning("Secure Boot is not enabled. Skipping shell presence check.")
        return ModuleResult.WARNING

    mark = self._uefi.get_EFI_variable('ShellMark', SHELLMARK_GUID)
    if not mark:
        self.logger.log_warning("ShellMark EFI variable not found. Run this module with 'setup'
first.")
        return ModuleResult.WARNING

    if mark[0] == 0x00:
        self.logger.log_passed("ShellMark is 0x00: EFI Internal Shell did not run. Secure Boot
correctly blocked it.")
        return ModuleResult.PASSED
    else:
        self.logger.log_failed("ShellMark is 0x01: EFI Internal Shell ran despite Secure Boot
being enabled! System is VULNERABLE (CVE-2023-48733).")
        return ModuleResult.FAILED

```

شکل ۴۲: بررسی فعال بودن Secure Boot و مقدار ShellMark

زمانی که ماژول بدون آرگومان اجرا شود تابع `check` صدا زده می‌شود. اگر متغیر `ShellMark` تعریف نشده باشد از کاربر خواسته می‌شود که ماژول را با آرگومان `setup` صدا بزند. در تابع `setup` فعالیت‌های آماده‌سازی ری بوت انجام می‌شود و از کاربر خواسته می‌شود که ری بوت کند. اگر شل بوت شود مقدار `ShellMark` عوض می‌شود و دوباره سیستم ری بوت می‌شود. اگر شل نتواند بوت شود مستقیماً OS اجرا می‌شود. در اجرای بعدی ماژول از روی مقدار `ShellMark` نتیجه به کاربر اعلام می‌شود.

۱۶.۲ نتایج اجرای ماژول OVMF

نتایج اجرای ماژول را در OVMF با سه نسخه 2022.08-1، 2022.11-6+deb12u2 و 2025.11-4 آورده‌ایم.

```
[*] Running module: chipsec.modules.sut.secureboot_fullshell
WARNING: ShellMark EFI variable not found. Run this module with 'setup' first.

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed      0.010
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed      0:
[CHIPSEC] Modules information 0:
[CHIPSEC] Modules failed      0:
[CHIPSEC] Modules with warnings 1:
WARNING: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total        1
[CHIPSEC] *****
```

شکل ۴۳: اجرای اول check در نسخه آسیب‌پذیر 2022.08-1. هنوز setup اجرا نشده است.

```
[*] Running module: chipsec.modules.sut.secureboot_fullshell
[*] Module arguments (1):
['setup']
Found EFI Internal Shell boot option: Boot0007
Wrote /boot/efi/startup.nsh
Set ShellMark EFI variable to 0x00
Next boot set to Boot0007 (EFI Internal Shell)
Please reboot now. After the system comes back up, run this module without 'setup' to check the result.

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed      0.128
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed      0:
[CHIPSEC] Modules information 0:
[CHIPSEC] Modules failed      0:
[CHIPSEC] Modules with warnings 1:
WARNING: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total        1
[CHIPSEC] *****
```

شکل ۴۴: اجرای اول setup در نسخه 2022.08-1

```
[+] loaded chipsec.modules.sut.secureboot_fullshell
[*] running loaded modules ..

[*] Running module: chipsec.modules.sut.secureboot_fullshell
[-] FAILED: ShellMark is 0x01: EFI Internal Shell ran despite Secure Boot being enabled! System is VULNERABLE (CVE-2023-48733).

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed      0.012
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed      0:
[CHIPSEC] Modules information 0:
[CHIPSEC] Modules failed      1:
[-] FAILED: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules with warnings 0:
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total        1
[CHIPSEC] *****
```

شکل ۴۵: اجرای دوم check پس از ریست در نسخه 2022.08-1 مقدار ShellMark تغییر کرده و آسیب‌پذیری به درستی شناسایی شده است.

```
[+] loaded chipsec.modules.sut.secureboot_fullshell
[*] running loaded modules ..

[*] Running module: chipsec.modules.sut.secureboot_fullshell
[*] Module arguments (1):
['setup']
Found EFI Internal Shell boot option: Boot0007
Wrote /boot/efi/startup.nsh
Set ShellMark EFI variable to 0x00
Next boot set to Boot0007 (EFI Internal Shell)
Please reboot now. After the system comes back up, run this module without 'setup' to check the result.

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed      0.118
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed      0:
[CHIPSEC] Modules information 0:
[CHIPSEC] Modules failed      0:
[CHIPSEC] Modules with warnings 1:
WARNING: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total      1
[CHIPSEC] *****
```

شکل ۴۶: اجرای setup در اولین نسخه تصحیح شده 2022.11-6+deb12u2

```
[+] loaded chipsec.modules.sut.secureboot_fullshell
[*] running loaded modules ..

[*] Running module: chipsec.modules.sut.secureboot_fullshell
[+] PASSED: ShellMark is 0x00: EFI Internal Shell did not run. Secure Boot correctly blocked it.

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed      0.008
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed      1:
[+] PASSED: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules information 0:
[CHIPSEC] Modules failed      0:
[CHIPSEC] Modules with warnings 0:
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total      1
[CHIPSEC] *****
```

شکل ۴۷: اجرای check در نسخه 2022.11-6+deb12u2 پس از ریوت. مقدار ShellMark تغییر نکرده و در نتیجه آسیب‌پذیر نیست.


```
[+] loaded chipsec.modules.sut.secureboot_fullshell
[*] running loaded modules ..

[*] Running module: chipsec.modules.sut.secureboot_fullshell
[*] Module arguments (1):
['setup']
[+] No 'EFI Internal Shell' boot option found. System is not vulnerable.

[CHIPSEC] ***** SUMMARY *****
[CHIPSEC] Time elapsed          0.034
[CHIPSEC] Modules failed to run 0:
[CHIPSEC] Modules passed       1:
[+] PASSED: chipsec.modules.sut.secureboot_fullshell
[CHIPSEC] Modules information  0:
[CHIPSEC] Modules failed       0:
[CHIPSEC] Modules with warnings 0:
[CHIPSEC] Modules not applicable 0:
[CHIPSEC] Modules total        1
[CHIPSEC] *****
```

شکل ۴۸: اجرای setup در آخرین نسخه 4-2025.11.2025. گزینه‌ای برای شل وجود ندارد و در همین مرحله ماژول پذیرفته می‌شود.

۳ نتیجه‌گیری

در این پروژه ابتدا با ساختار و نحوه اجرای CHIPSEC آشنا شدیم و ماژول‌های پیش‌فرض آن را روی سخت‌افزار واقعی اجرا کردیم. در ادامه معماری این چارچوب، لایه‌های HAL و OS Helper و نحوه دسترسی به متغیرهای EFI بررسی شد. همچنین با دامپ گرفتن از firmware و استفاده از ابزارهای UEFITool و ghidra، اجزای آن و کدهای مرتبط با Secure Boot را مطالعه کردیم. این بررسی‌ها زمینه لازم را برای شناخت آسیب‌پذیری‌های مبتنی بر متغیرهای NVRAM و پیاده‌سازی ماژول‌های سفارشی فراهم کرد.

در بخش عملی، خانواده‌ای از آسیب‌پذیری‌های لپ‌تاپ‌های لنوو و به طور خاص CVE-2022-3432 بررسی شد. آزمایش انجام‌شده روی دستگاه نشان داد که وجود متغیر L05SecBootSmm با مقدار صفر در این مدل باعث غیرفعال شدن Secure Boot نمی‌شود. علاوه بر این، محیطی برای اجرای CHIPSEC با Secure Boot فعال آماده شد و اجرای آن در QEMU و OVMF نیز مورد بررسی قرار گرفت. در این محیط، آسیب‌پذیری CVE-2023-48733 با چند نسخه متفاوت از OVMF آزمایش شد و روش تشخیص بر اساس بوت شدن واقعی EFI Shell توانست نسخه آسیب‌پذیر را از نسخه‌های تصحیح‌شده تشخیص دهد.

خروجی کد این پروژه شامل دو ماژول check_backdoor_vars و secureboot_fullshell است. ماژول اول متغیرهای EFI مرتبط با درپشتی‌های شناخته‌شده لنوو را به صورت غیرفعال بررسی می‌کند و ماژول دوم امکان بوت شدن FullShell را هنگام فعال بودن Secure Boot می‌آزماید. در مجموع، پروژه علاوه بر بررسی فنی آسیب‌پذیری‌ها، دو روش قابل اجرا برای تشخیص آن‌ها در سخت‌افزار واقعی و محیط مجازی ارائه کرد و نتایج هر دو ماژول نیز در محیط‌های مورد نظر آزمایش و تأیید شد.

- [1] CVE.org, “CVE-2022-3432.” <https://www.cve.org/CVERecord?id=CVE-2022-3432>
- [2] Lenovo, “Lenovo Notebook BIOS Vulnerabilities – LEN-94952.” https://support.lenovo.com/us/en/product_security/LEN-94952
- [3] ESET Research, “When ‘secure’ isn’t secure at all: High-impact UEFI vulnerabilities discovered in Lenovo consumer laptops.” <https://www.welivesecurity.com/2022/04/19/when-secure-isnt-secure-uefi-vulnerabilities-lenovo-consumer-laptops/>
- [4] ESET Research, “Details of CVE-2022-3430, CVE-2022-3431 and CVE-2022-3432.” <https://x.com/ESETresearch/status/1590279809841913858>
- [5] Nikolaj Schlej, “UEFITool.” GitHub repository. <https://github.com/longsoft/uefitool>
- [6] Digital Cybersecurity Group, “UEFI Surveyor.” GitHub repository. https://github.com/dcg-red-team/UEFI_Surveyor
- [7] ESET Research, “Details of Lenovo UEFI vulnerabilities CVE-2021-3970, CVE-2021-3971 and CVE-2021-3972.” <https://x.com/ESETresearch/status/1516365339080146944>
- [8] Lenovo, “Lenovo Notebook BIOS Vulnerabilities – LEN-73440.” https://support.lenovo.com/us/en/product_security/LEN-73440
- [9] National Vulnerability Database, “CVE-2023-48733 Detail.” <https://nvd.nist.gov/vuln/detail/CVE-2023-48733>
- [10] Tenable, “Debian DSA-5614-1: edk2 – security update.” <https://www.tenable.com/plugins/nessus/190574>
- [11] Debian Snapshot, “Source package edk2, version 2022.08-1.” <https://snapshot.debian.org/package/edk2/2022.08-1/>
- [12] Debian Snapshot, “Source package edk2, version 2022.11-6+deb12u2.” <https://snapshot.debian.org/package/edk2/2022.11-6%2Bdeb12u2/>